Sorting

Algoritmos y estructuras de datos 2c2026

by Damy



Temas de hoy

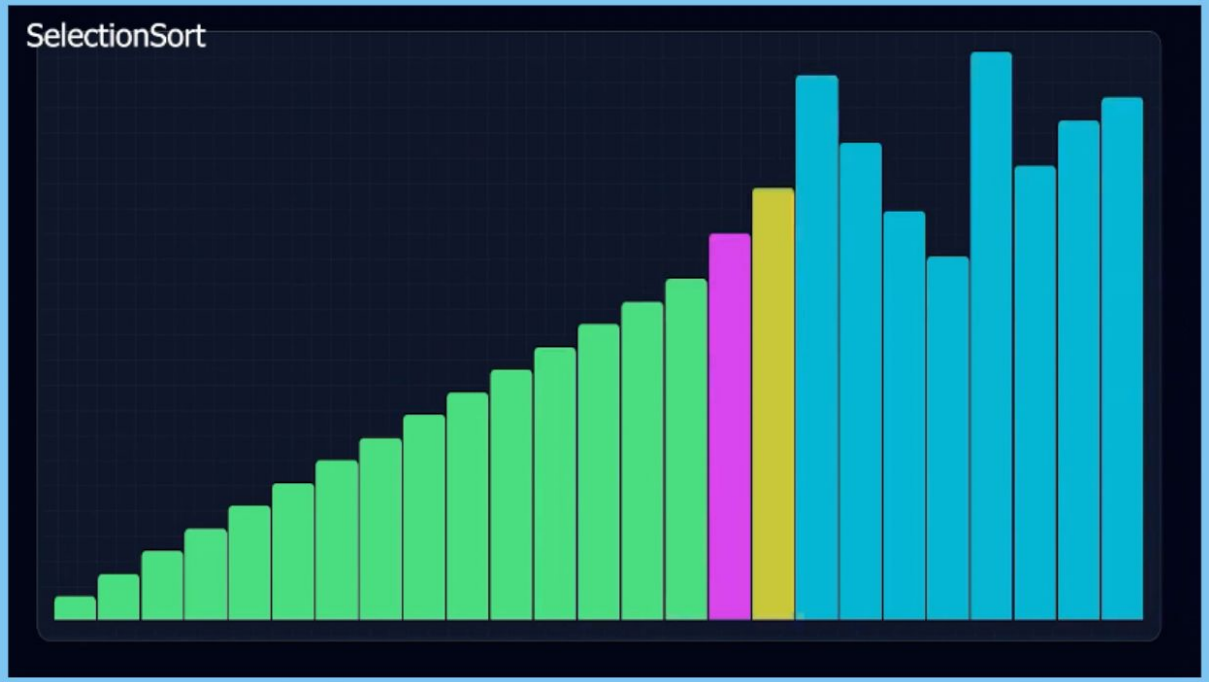
- Repaso de estabilidad + algoritmos de la teórica
- Ordenar en tiempo lineal: Counting, bucket y radix
- Ejercicios

Estabilidad

Un algoritmo de ordenamiento es estable si no modifica el orden relativo de los elementos que tienen igual valor según nuestro criterio de ordenamiento.

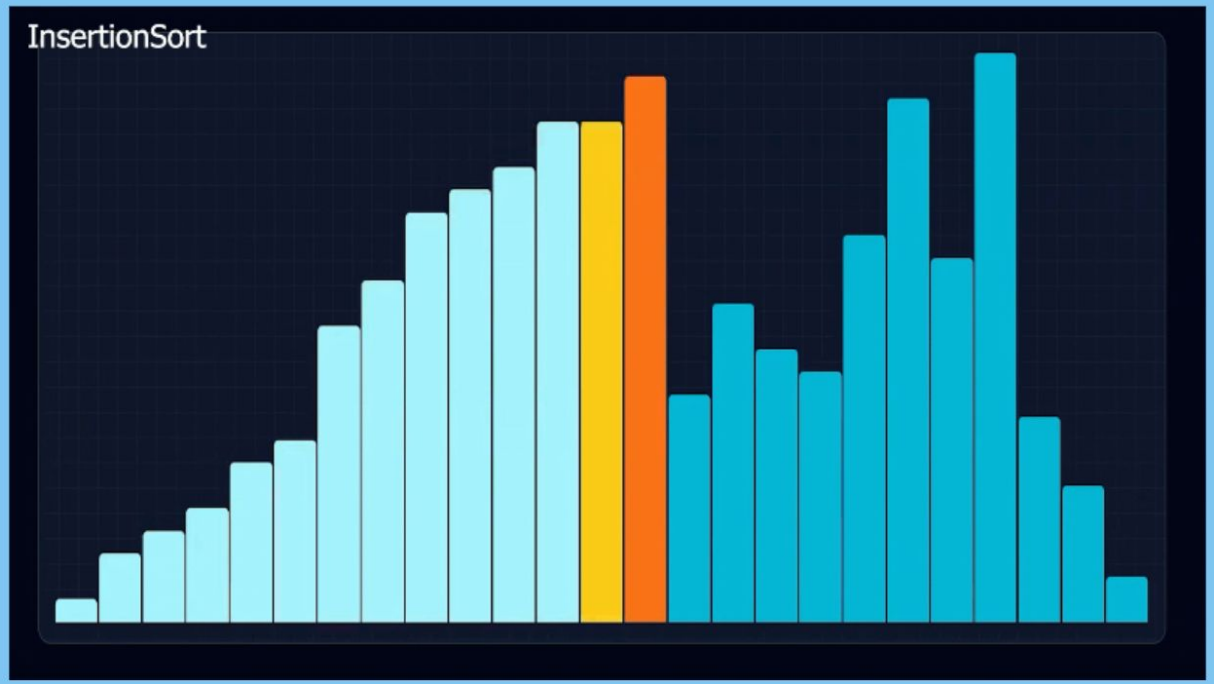
Algoritmo	Complejidad peor caso	¿Es estable?
SelectionSort		
InsertionSort		
QuickSort		
MergeSort		
HeapSort		

Selection Sort



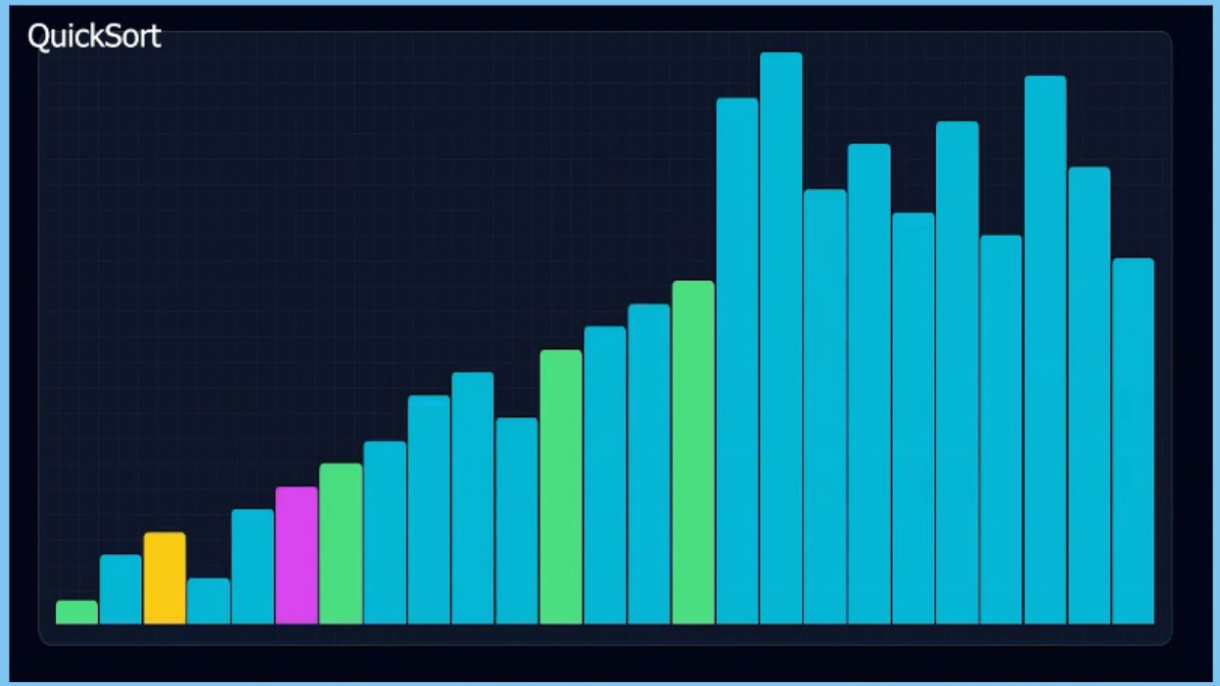
Algoritmo	Complejidad peor caso	¿Es estable?
SelectionSort	$O(n^2)$	No
InsertionSort		
QuickSort		
MergeSort		
HeapSort		

Insertion Sort



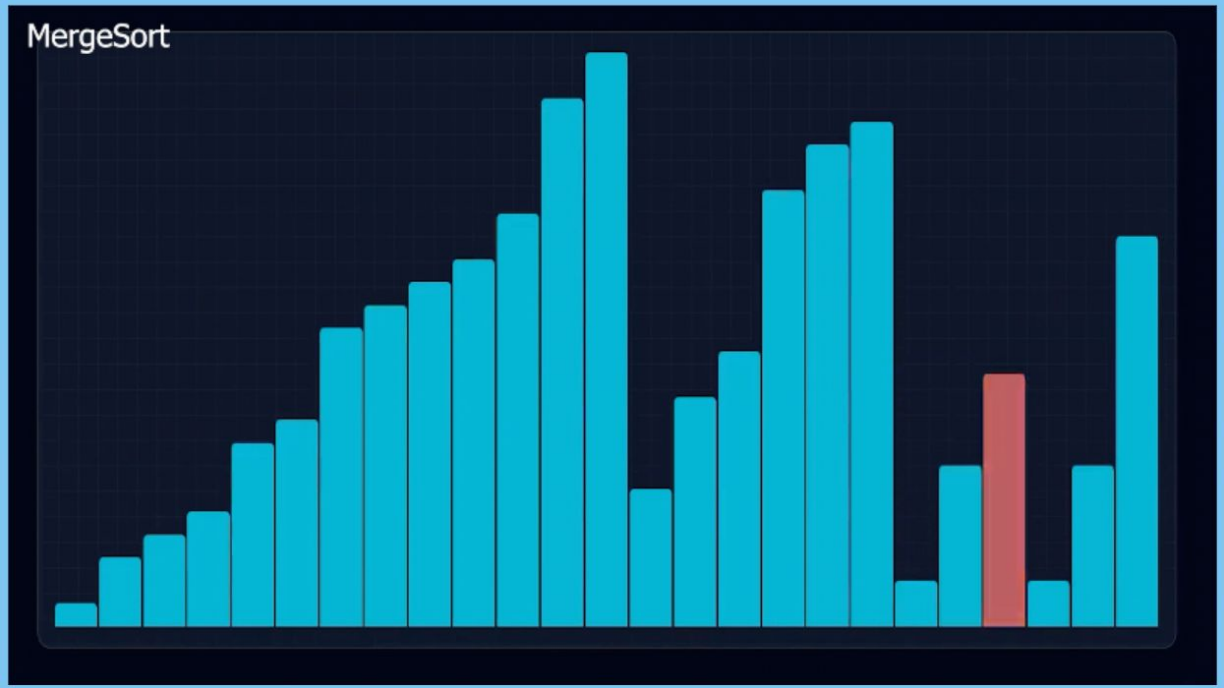
Algoritmo	Complejidad peor caso	¿Es estable?
SelectionSort	$O(n^2)$	No
InsertionSort	$O(n^2)$	Si
QuickSort		
MergeSort		
HeapSort		

Quick Sort



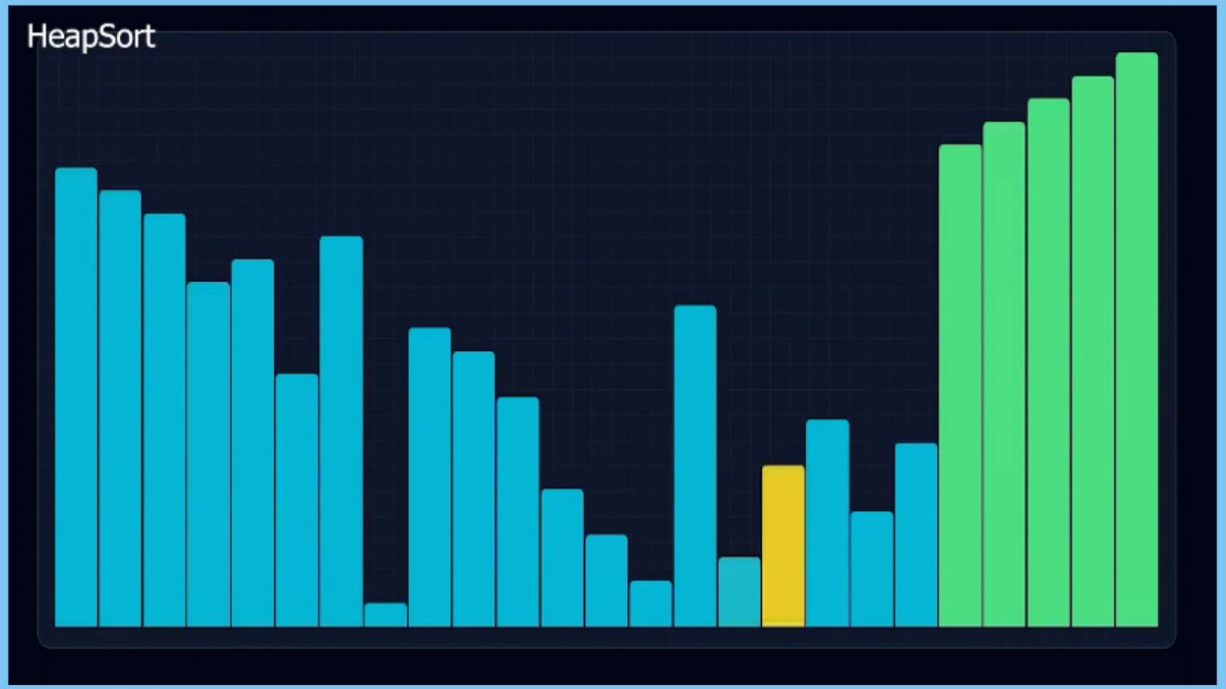
Algoritmo	Complejidad peor caso	¿Es estable?
SelectionSort	$O(n^2)$	No
InsertionSort	$O(n^2)$	Si
QuickSort	$O(n^2)$	No
MergeSort		
HeapSort		

Merge Sort



Algoritmo	Complejidad peor caso	¿Es estable?
SelectionSort	$O(n^2)$	No
InsertionSort	$O(n^2)$	Si
QuickSort	$O(n^2)$	No
MergeSort	$O(n*log(n))$	Si
HeapSort		

Heap Sort



Algoritmo	Complejidad peor caso	¿Es estable?
SelectionSort	$O(n^2)$	No
InsertionSort	$O(n^2)$	Si
QuickSort	$O(n^2)$	No
MergeSort	O(n*log(n))	Si
HeapSort	O(n*log(n))	No

Algoritmo	Complejidad* peor caso	¿Es estable?
SelectionSort	$O(n^2)$	No
InsertionSort	$O(n^2)$	Si
QuickSort	$O(n^2)$	No
MergeSort	O(n*log(n))	Si
HeapSort	O(n*log(n))	No

*Asumiendo que comparar dos elementos es O(1)



Animaciones Sorting

Click aquí

Sintaxis

```
Proc xSort(A: array<T>, fVaPrimero: T x T -> T)
```

- Los algoritmos de ordenamiento por comparación reciben un array (o un vector) y una función de comparación.
- El arreglo/vector pasado como parámetro se modifica (es inout).
- La función de comparación recibe dos elementos del mismo tipo del arreglo y devuelve el que va primero según nuestro criterio de orden.
- No hace falta implementar dicha función, con ponerle un nombre declarativo o escribir un comentario del criterio es suficiente.

Ejemplos

```
//B es array de enteros
mergeSort(B, deMenorAMayor)

//C es vector de tuplas<int, string>
heapSort(C, fVaPrimero) // Ordena de forma decreciente según primer elemento de la tupla
```

Ejemplo: Estabilidad

Dado un arreglo A de tuplas<int, int> quiero ordenar de forma creciente según primer elemento de la tupla, y en caso de empate, ordenar de forma creciente según primer elemento

Criterio principal:
1er elemento
Criterio de desempate:
2do elemento

A: [<5, 4>, <1, 7>, <2, 6>, <5, 3>, <6, 2>]

Ordeno primero del criterio de desempate (de forma estable o no estable)

[<6, 2>, <5, 3>, <5, 4>, <2, 6>, <1, 7>]

Ordeno por criterio principal de forma estable para preservar la posición relativa de los elementos iguales

[<1, 7>, <2, 6>, <5, 3>, <5, 4>, <6, 2>]

QuickSort(A, fVaPrimero) // Ordena de forma creciente según segundo elemento de la tupla

insertionSort(A, fVaPrimero') // Ordena (De forma estable) de forma creciente según primer elemento de la tupla

Teorema lower-bound

En los algoritmos de ordenamiento por comparaciones, la complejidad del peor caso es $\Omega(n \log n)$

¿Podríamos mejorarla si tenemos información del contenido de los arreglos de antemano?

Más algoritmos:
Ordenar en tiempo lineal



Algoritmo	Complejidad	¿Es estable?	Obs
CountingSort			
BucketSort			
RadixSort			

Counting Sort Nat

COUNTING-SORT-NAT(A: arreglo(Nat), max:Nat):

```
1.  intervaloSize ← max + 1
2.  B ← arreglo de tamaño intervaloSize
3.  for i ← [0, length(B)-1] do // O(max)
4.      B[i] ← 0
5.  end for
    //cuento la cantidad de apariciones de cada elemento
6.  for i ← [0..length(A)-1] do // O(n)
7.     B[A[i]] ← B[A[i]] +1
8.  end for
    //inserto en A la cantidad de apariciones de cada elemento
9.  indexA ← 0
10. for i ← [0..length(B)-1] do // O(max+n)
11.     for j ← [1..B[i]] do
12.         A[indexA] ← i
13.         indexA ← indexA + 1
14.     end for
15. end for
```

Counting Sort: Generalización

```
COUNTING-SORT(A: arreglo(int), min: int, max:int):

1.  intervaloSize ← max - min + 1
2.  B ← arreglo de tamaño intervaloSize
3.  for i ← [0, length(B)-1] do // O(max-min)
4.      B[i] ← 0
5.  end for
    //cuento la cantidad de apariciones de cada elemento
6.  for i ← [0..length(A)-1] do // O(n)
7.     B[A[i]-min] ← B[A[i]-min] +1
8.  end for
    //inserto en A la cantidad de apariciones de cada elemento
9.  indexA ← 0
10. for i ← [0..length(B)-1] do // O(max-min + n)
11.     for j ← [1..B[i]] do
12.         A[indexA] ← i + min
13.         indexA ← indexA + 1
14.     end for
15. end for
```

Algoritmo	Complejidad	¿Es estable?	Obs
CountingSort	O(n+(max-min))	No: Destruye los datos	Rango acotado: O(n)
BucketSort			
RadixSort			

Ejemplos

```
//B es array de números acotados entre  10 y 1000. Deseo ordenarlos crecientemente

countingSort(B, 10, 1000) // O(n+(1000-10)) = O(n)
```

BucketSort: Motivación

Se desea ordenar un arreglo de tuplas <cargo, nombre> (ambos strings) en orden jerárquico respecto a los cargos docentes, es decir, que primero esten los Profesores (“Profe”), seguidos de los Jefes de trabajos prácticos (“JTP”) y luego los ayudantes (“Ay”).

Idea:

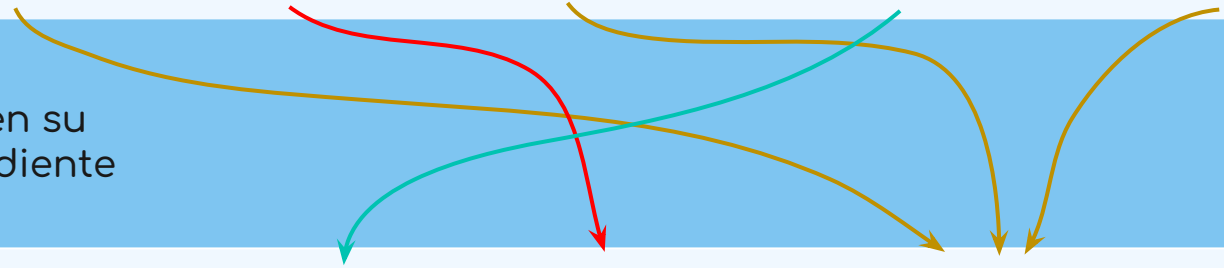
Entrada: [<"Ay", "Luis">, <"JTP", "Diego">, <"Ay", "Valen">, <"Profe", "Sebas">, <"Ay", "Nacho">]

Arreglo auxiliar: [Colección Profes, Colección JTPs, Colección ayudantes]

Idea:

Entrada: [<"Ay", "Luis">, <"JTP", "Diego">, <"Ay", "Valen">, <"Profe", "Sebas">, <"Ay", "Nacho">]

Meto los elementos en su colección correspondiente



Arreglo auxiliar: [Colección Profes, Colección JTPs, Colección ayudantes]

Obs: Las tuplas de mi arreglo original quedaron dentro de las colecciones, si las voy recorriendo de izquierda a derecha obteniendo cada uno de sus elementos, me los darán en el orden que quiero!

¿Con qué estructura de datos podamos representar dicha colección para que podamos insertar rápido?
Lista enlazada O(1)!

Idea:

Entrada: [<"Ay", "Luis">, <"JTP", "Diego">, <"Ay", "Valen">, <"Profe", "Sebas">, <"Ay", "Nacho">]

Arreglo auxiliar de L.Es: [L.E vacia Profes, L.E vacia JTP, L.E vacia Ay]

Idea:

Entrada: [<"Ay", "Luis">, <"JTP", "Diego">, <"Ay", "Valen">, <"Profe", "Sebas">, <"Ay", "Nacho">]

Meto los elementos en su Lista enlazada correspondiente

Arreglo auxiliar de L.Es: [<"Profe", "Sebas">, <"JTP", "Diego">, <"Ay", "Luis">]

<"Ay", "Valen">

<"Ay", "Nacho">

Concateno las listas:

<"Profe", "Sebas"> → <"JTP", "Diego"> → <"Ay", "Luis"> → <"Ay", "Valen"> → <"Ay", "Nacho">

¿Cómo puedo saber a qué lista enlazada va cada elemento?

A partir de una "función de asignación" (Llamémosla h) que te diga la posición del arreglo auxiliar en donde hay que agregarlo.

```
Proc h(elem: Tupla<String, String>): int
1.  int res;
2.  if(elem[0] == "Profe") then
3.      res = 0;
4.  end if
5.  if(elem[0] == "JTP") then
6.      res = 1;
7.  end if
8.  if(elem[0] == "Ay") then
9.      res = 2;
10. end if
11. return res;
```



Bucket Sort

```
BUCKETSORT(A: Arreglo(T), M: int, h: T -> int)
1.    n ← length(A)
2.    B ← arreglo de M listas enlazadas vacías // O(M)
3.  //inserta los elementos en su bucket correspondiente según h
4.    for i ← [0..n-1] do // O(n * costoAsignar)
5.        Insertar A[i] en la lista B[h(A[i])]
6.    end for
7.    Concatenar B[0], B[1], ..., B[M-1] // O(n+M)
8.  //Recorre la concatenación y pone los elementos en A
```

- Obs: La imagen de la función h es desde 0 hasta M-1
- No hace falta escribir la función h al llamar al algoritmo, pero sí explicarlo con palabras

Algoritmo	Complejidad	¿Es estable?	Obs
CountingSort	O(n+(max-min))	No: Destruye los datos	Rango acotado: O(n)
BucketSort	O(n*costoAsignar + M)	Si	Función h en O(1) y categorias acotadas: O(n)
RadixSort			

Ejemplos

```
//B es array de números acotados entre  10 y 1000. Deseo ordenarlos crecientemente

countingSort(B, 10, 1000) // O(n+(1000-10)) = O(n)

//C es array de tuplas cuya primer componente puede ser "gato" o "perro", quiero ordenar de tal manera que siempre queden los gatos primero y luego los perros

bucketSort(C, 2, h)
// El primer bucket tiene los elementos cuyo 1er componente de la tupla es "gato" y el segundo bucket cuyo 1er componente es "perro".
// Es decir: h(<"gato", ...>) = 0 y h(<"perro">, ...>) = 1, claramente asignar a cada bucket es O(1) -> O(n+2) = O(n)
```

RadixSort: Motivación

Deseo ordenar un arreglo A de strings cuya máxima cantidad de caracteres puede ser 3

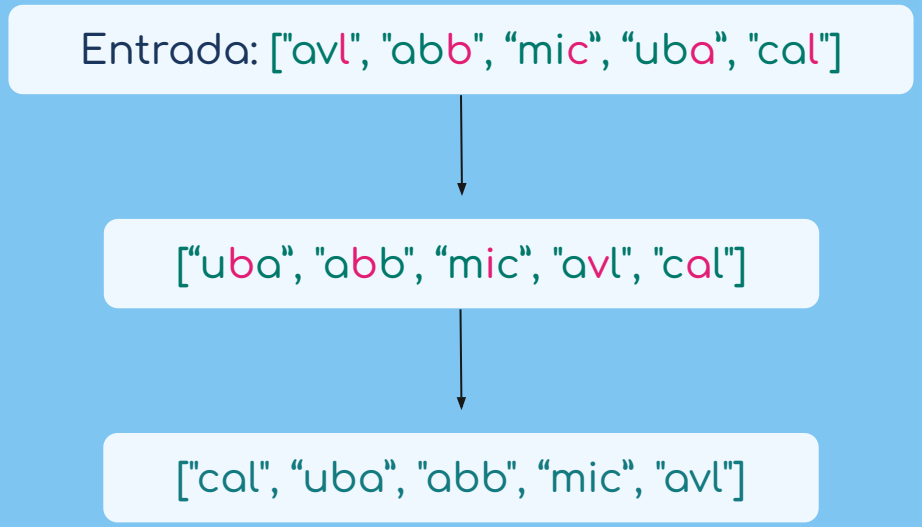
Entrada: ["avl", "abb", “mic“, “uba“, "cal"]

idea:

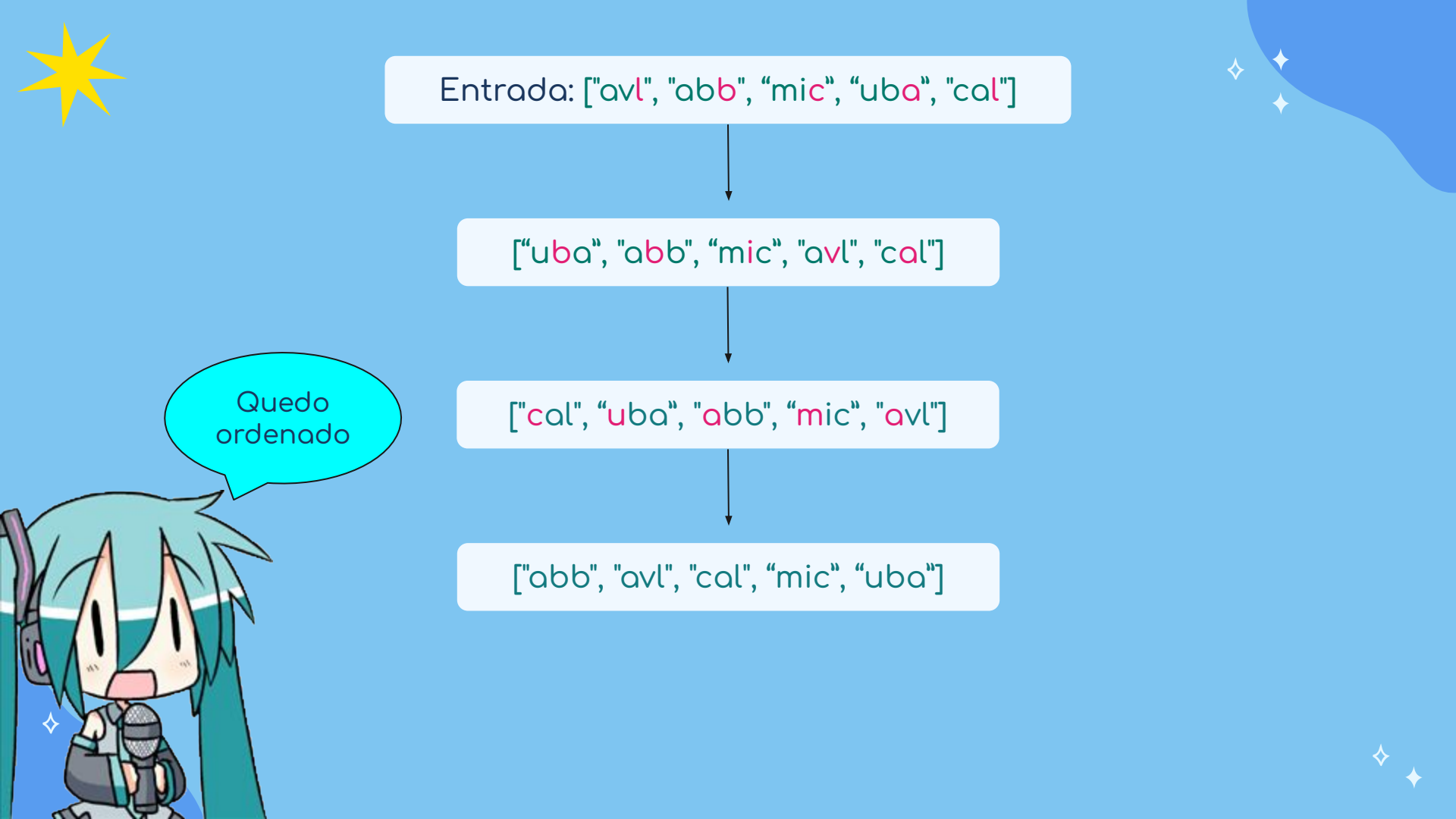
Aprovecho la estabilidad y ordeno del carácter menos significativo al más significativo (de derecha a izquierda) de forma estable con bucket Sort

Entrada: ["avl", "abb", “mic“, “uba“, "cal"]

[“uba“, "abb", “mic“, "avl", "cal"]


Entrada: ["avl", "abb", “mic“, “uba“, "cal"]
[“uba“, "abb", “mic“, "avl", "cal"]
["cal", “uba“, "abb", “mic“, "avl"]

Entrada: ["avl", "abb", “mic“, “uba“, "cal"]



["uba“, "abb", “mic“, "avl", "cal"]

["cal", “uba“, "abb", “mic“, "avl"]

["abb", "avl", "cal", “mic“, “uba“]

Radix Sort

```
RADIXSORT(A: Arreglo(T), d: int)
1.  //del menos significativo al más significativo
2.  for i ← [1..d] do // O(d*n)
3.      //En nuestro caso siempre con bucketSort
4.      Ordenar el arreglo A según el dígito i, en forma estable
5.  end for
```

- Sirve para ordenar arreglos de caracteres, números y tuplas*, ordenando desde el “dígito” menos significativo al más significativo con bucketSort aprovechando la estabilidad del mismo
- Obs: Los “dígitos” deben ser un alfabeto acotado (Letras del abecedario, números del 0 al 9, etc). Pues bucketSort requiere dicha cota para poder ser en O(n).
- *Por ejemplo tuplas cuyo primer componente sean dígitos

Algoritmo	Complejidad peor caso	¿Es estable?	Obs
CountingSort	O(n+(max-min))	No: Destruye los datos	Rango acotado: O(n)
BucketSort	O(n*costoAsignar + M)	Si	Función h en O(1) y categorias acotadas: O(n)
RadixSort	O(n*d)	Si	Cantidad de dígitos acotada sobre alfabeto finito: O(n)

Ejemplos

```
//B es array de números acotados entre  10 y 1000. Deseo ordenarlos crecientemente

countingSort(B, 10, 1000) // O(n+(1000-10)) = O(n)

//C es array de tuplas cuya primer componente puede ser "gato" o "perro", quiero ordenar de tal manera que siempre queden los gatos primero y luego los perros

bucketSort(C, 2, h)
// El primer bucket tiene los elementos cuyo 1er componente de la tupla es "gato" y el segundo bucket cuyo 1er componente es "perro".
// Es decir: h(<"gato", ...>) = 0 y h(<"perro">, ...>) = 1, claramente asignar a cada bucket es O(1) -> O(n+2) = O(n)

//D es un arreglo alfanumérico que quiero ordenar en orden lexicográfico, tiene como mucho 1000 caracteres:

radixSort(D, 1000) // O(n*1000) = O(n)
```

Ejercicios



Ejercicio 18. Dado un arreglo de pares <estudiante, nota>, queremos devolver un arreglo de pares <estudiante,promedio>, ordenado en forma descendente por promedio y, en caso de empate, por número de libreta.

Los estudiantes se representan con un número de libreta (puede considerar que es un número de a lo sumo 10 dígitos), y la nota es un número entre 1 y 100. La cantidad de estudiantes se considera no acotada.

Se pide dar un algoritmo que resuelva el problema con complejidad O(n + m log m), donde n es la cantidad total de notas (es decir, el tamaño del arreglo de entrada), y m es la cantidad total de estudiantes. Ejemplo:

Entrada: <12395, 72>, <45615, 81>, <12395, 94>, <45615, 100>, <78920, 81>, <12395, 90>, <45615, 71>, <78920, 50>
Salida: <12395, 85.3333>, <45615, 84>, <78920, 65.5>

a) Describa cada etapa del algoritmo con sus palabras, y justifique por qué cumple con las complejidades pedidas.

b) Escriba el algoritmo en pseudocódigo. Puede utilizar (sin reescribir) todos los algoritmos y estructuras de datos vistos en clase

Intervalo



Ejercicio de Parcial. Se está por realizar un gran censo de perros en la Ciudad de Buenos Aires. La información recaudada llegará en forma de un arreglo de tuplas <Nombre, Raza, Edad, Peso>. Las razas son *strings* de largo acotado *L*, pero no sabemos cuántas razas pueden existir, aunque sí sabemos que entre ellas está la mejor raza de todas: raza Perro. Las edades serán enteros positivos entre 1 y 20 y los pesos serán de tipo float.

Proponga un algoritmo que organice la información en dos arreglos:

- Uno con los perros raza Perro ordenados por edad y, en caso de empate, por peso, ambos en forma ascendente
- Otro arreglo con la información de las razas y la cantidad de perros de cada una en forma de tuplas <Raza, int>, ordenadas en forma decreciente por la cantidad y, en caso de empate, alfabéticamente por la raza

La complejidad temporal del algoritmo debe ser O(n + p*log(p)), donde *n* es el total de perros y *p* el total de perros raza Perro.
Ejemplo Si recibimos:
[<Nulus, Perro, 4, 15.3>, <Joe, Cocker, 14, 8.0>, <Lady, Caniche, 10, 6.1>, <Dobby, Perro, 7, 18.3>, <Res, Dogo, 3, 50.0>, <Haru, Perro, 7, 30.0>, <Clementina, Perro, 14, 10.0>, <Choqui, Caniche, 16, 5.0>]

Deberíamos devolver:
P: [<Nulus, 4, 15.3>, <Dobby, 7, 18.3>, <Haru, 7, 30.0>, <Clementina, 14, 10.0>]
R: [<Caniche, 2>, <Cocker, 1>, <Dogo, 1>]

Extra

Ejercicio 12. Se desea ordenar los datos generados por un sensor industrial que monitorea la presencia de una sustancia en un proceso químico. Cada una de estas mediciones es un número entero positivo. Dada la naturaleza del proceso se sabe que, dada una secuencia de n mediciones, a lo sumo $\lfloor \sqrt{n} \rfloor$ valores están fuera del rango [20, 40].

Proponer un algoritmo O(n) que permita ordenar ascendentemente una secuencia de mediciones y justificar la complejidad del algoritmo propuesto.

Consultas

